Route Resilience System

Urban Road-Network Resilience Analysis Platform

A graph-theoretic approach to identifying critical junctions,
simulating disaster scenarios, and planning resilient interventions
for urban road networks using satellite imagery and geospatial data.

Technology Stack: Python · FastAPI · Next.js · NetworkX · OSMnx · PyTorch

Table of Contents

1. Project Overview
2. Problem Statement
3. System Architecture & Pipeline
4. Technology Stack
5. Directory Structure
6. Backend — Core Modules
7. Frontend — User Interface Pages
8. API Reference
9. Graph Metrics & Key Statistics
10. Disaster Simulation Engine
11. ML / AI Components
12. External Integrations
13. Feature Coverage Matrix
14. Setup & Quick Start Guide
15. Environment Variables
16. Future Scope

1. Project Overview

The **Route Resilience System** is a full-stack web application that performs **graph-theoretic resilience analysis** on urban road networks. It ingests real-world geospatial data — street graphs from OpenStreetMap, satellite imagery, population density, and digital elevation models — to build a weighted road graph and then quantifies how robust the network is against disruptions such as natural disasters, flooding, landslides, and infrastructure failures.

The platform enables city planners, disaster-management authorities, and researchers to:

- Identify **critical junctions** whose failure would fragment the road network.
- Run **disaster simulations** (seismic, flood, cascade failure) and observe network degradation.
- **Simulate counterfactual interventions** — bridge additions, bypass roads — and measure their impact.
- Visualise **criticality heatmaps** on interactive Leaflet maps.
- Generate **automated reports** with resilience scores and recommendations.
- Interact with an **AI Copilot** (Groq LLM) for natural-language Q&A; about the network.

2. Problem Statement

Urban road networks are the backbone of a city's logistics, emergency response, and daily commute. Yet, most cities lack tooling to answer a simple question: *"If a key junction or road segment fails, how badly is the network affected, and what can we do about it?"*

This project addresses **ISRO NNRMS Hackathon 2026 — Problem Statement PS4**: *Urban Road Network Resilience Analysis using Satellite Imagery and Graph Theory*. The objective is to extract road networks from satellite imagery, convert them into weighted graphs, and apply centrality and resilience metrics to identify vulnerabilities.

3. System Architecture & Pipeline

The system follows a **multi-stage pipeline** architecture:

Stage	Component	Description
1	Data Ingestion	Fetch road graph from OpenStreetMap via OSMnx / Overpass API; ingest satellite imagery, DEM tiles, po
2	Road Extraction (ML)	Optional U-Net segmentation model extracts road pixels from satellite imagery and skeletonizes them into
3	Graph Construction	Build a NetworkX weighted graph: nodes = intersections, edges = road segments weighted by length, spe
4	Metric Computation	Compute centrality scores (betweenness, closeness, degree, eigenvector), MST fraction, algebraic conne
5	Criticality Analysis	Rank junctions by composite criticality; identify bridges and articulation points.

Stage	Component	Description
6	Disaster Simulation	Remove nodes/edges matching a hazard scenario; measure fragmentation, detour lengths, population isc
7	Counterfactual Healing	Simulate adding bypass edges; re-measure metrics; compute resilience gain.
8	Visualisation & Reports	Serve interactive maps, charts, and downloadable PDF/HTML reports via the web dashboard.

High-Level Data Flow

```
Satellite Imagery / OSM Data
↓
Road Extraction (U-Net) / OSMnx Graph Download
↓
Graph Construction (NetworkX weighted graph)
↓
Centrality & Resilience Metrics
↓
Critical Junction Detection
↓
Disaster Simulation ←→ Counterfactual Interventions
↓
Dashboard + Reports + AI Copilot
```

4. Technology Stack

Category	Technology
Frontend Framework	Next.js 14.2.3 (React 18)
UI / Styling	Tailwind CSS, Radix UI, shadcn/ui components
Maps	Leaflet + react-leaflet
Charts	Recharts
Language (Frontend)	TypeScript
Backend Framework	FastAPI (Python)
Graph Library	NetworkX
Geospatial	OSMnx, GeoPandas, Shapely, pyproj, rasterio
ML / DL	PyTorch, torchvision, segmentation-models-pytorch (optional)
LLM Integration	Groq API (Llama / Mixtral via groq-sdk)
HTTP Server	Uvicorn (ASGI)
Data Formats	GeoJSON, GeoPackage (GPKG), GeoPickle, CSV
Package Manager	pip (backend), npm (frontend)
Version Control	Git + GitHub

5. Directory Structure

```
route-resilience/
■■■ backend/
■ ■■■ app/
■ ■ ■■■ main.py # FastAPI entry point, CORS, lifespan
■ ■ ■■■ api/ # API route modules
■ ■ ■ ■■■ graph.py # /graph/* endpoints
■ ■ ■ ■■■ simulation.py # /simulate/* endpoints
■ ■ ■ ■■■ accessibility.py # /accessibility/* endpoints
■ ■ ■ ■■■ copilot.py # /copilot/* endpoints
■ ■ ■ ■■■ reports.py # /reports/* endpoints
■ ■ ■ ■■■ bhuvan.py # /bhuvan/* endpoints
■ ■ ■ ■■■ segmentation.py # /segment/* endpoints
■ ■ ■■■ graph_pipeline/ # Core graph analysis
■ ■ ■ ■■■ graph_build.py # Graph construction & loading
■ ■ ■ ■■■ centrality.py # Centrality computations
■ ■ ■ ■■■ metrics.py # Network resilience metrics
■ ■ ■ ■■■ mst_healing.py # MST-based healing strategies
■ ■ ■ ■■■ skeletonize.py # Image → skeleton → graph
■ ■ ■ ■■■ fingerprint.py # Graph fingerprinting
■ ■ ■■■ simulation/ # Disaster simulation modules
■ ■ ■ ■■■ ablation.py # Node/edge ablation
■ ■ ■ ■■■ cascade.py # Cascade failure model
■ ■ ■ ■■■ resilience.py # Resilience score computation
■ ■ ■ ■■■ scenarios.py # Predefined disaster scenarios
■ ■ ■ ■■■ routing.py # Shortest-path routing
■ ■ ■ ■■■ fragility.py # Fragility curves
■ ■ ■ ■■■ recommendations.py # Automated recommendations
■ ■ ■ ■■■ population.py # Population impact analysis
■ ■ ■ ■■■ topography.py # Elevation-based analysis
■ ■ ■ ■■■ evacuation.py # Evacuation route planning
■ ■ ■ ■■■ equity.py # Equity-weighted analysis
■ ■ ■ ■■■ traffic_impact.py # Traffic flow impact
■ ■ ■ ■■■ temporal_degradation.py # Time-based degradation
■ ■ ■ ■■■ timeline.py # Event timeline tracking
■ ■ ■■■ integrations/ # External service clients
■ ■ ■ ■■■ bhuvan.py # ISRO Bhuvan tile proxy
■ ■ ■ ■■■ groq_client.py # Groq LLM integration
■ ■ ■ ■■■ nominatim.py # Geocoding via Nominatim
■ ■ ■ ■■■ overpass.py # Overpass API client
■ ■ ■■■ ml/ # Machine learning pipeline
■ ■ ■ ■■■ model.py # U-Net architecture
```

```
■ ■ ■ ■■■ inference.py # Model inference
■ ■ ■ ■■■ train.py # Training loop
■ ■ ■ ■■■ augmentations.py # Data augmentations
■ ■ ■ ■■■ explain.py # Grad-CAM explanations
■ ■ ■ ■■■ change_detection.py # Temporal change detection
■ ■ ■■■ reports/
■ ■ ■■■ generator.py # PDF/HTML report generation
■ ■■■ data/ # Generated data (gitignored)
■ ■■■ cache/ # Overpass API cache
■ ■■■ scripts/
■ ■ ■■■ download_data.py # Data download utility
■ ■■■ requirements.txt # Full dependency list
■ ■■■ requirements-core.txt # Core dependencies only
■■■ frontend/
■ ■■■ app/ # Next.js App Router pages
■ ■ ■■■ page.tsx # Dashboard (home page)
■ ■ ■■■ map/page.tsx # Criticality Heatmap
■ ■ ■■■ simulate/page.tsx # Disaster Simulation
■ ■ ■■■ analytics/page.tsx # Analytics dashboard
■ ■ ■■■ copilot/page.tsx # AI Copilot chat
■ ■ ■■■ reports/page.tsx # Report generation
■ ■ ■■■ explain/page.tsx # ML Explanation / Segmentation
■ ■■■ components/ # Reusable React components
■ ■■■ lib/ # Utility functions & API client
■ ■■■ package.json
■■■ experiments/ # Research experiment logs
■■■ .env.example # Environment variable template
■■■ start_demo.ps1 # Windows demo launcher
■■■ README.md
```

6. Backend — Core Modules

6.1 Graph Pipeline

The graph pipeline is the heart of the system. It handles building the road network graph from raw geospatial data and computing all resilience-related metrics.

Module	Description
graph_build.py	Loads the road network from OpenStreetMap via OSMnx. Supports bounding-box queries with configurable AO
centrality.py	Computes node-level centrality measures: betweenness centrality, closeness centrality, degree centrality, eige
metrics.py	Calculates global network metrics: algebraic connectivity (Fiedler value), global efficiency, average clustering c
mst_healing.py	Implements MST-based graph healing: identifies the minimum spanning tree of the damaged network, propose
skeletonize.py	Converts binary road segmentation masks into 1-pixel-wide skeletons using morphological thinning, then extra
fingerprint.py	Creates a compact 'fingerprint' of the graph topology — degree distribution, centrality histograms — for quick c

6.2 Simulation Engine

The simulation package models different disaster scenarios and quantifies their impact on the road network.

Module	Description
ablation.py	Removes a set of nodes or edges from the graph and measures resulting fragmentation.
cascade.py	Models cascade failure: removing one node overloads its neighbors, which may also fail, triggering a chain
resilience.py	Computes the resilience score — the fraction of network functionality retained after a disruption.
scenarios.py	Predefined disaster scenarios: earthquake, flood, landslide, targeted attack. Each scenario defines which n
routing.py	Computes shortest paths and detour analysis: how much longer do routes become when critical links are re
fragility.py	Fragility curve modeling for structural elements under varying hazard intensities.
population.py	Overlays population density data (WorldPop) to estimate how many people are affected by network disrupt
topography.py	Uses DEM (elevation) data to identify flood-prone low-lying areas and steep gradients that affect road vulne
evacuation.py	Plans evacuation routes from affected zones to safe shelters, considering road capacity and damage.
equity.py	Equity-weighted analysis: ensures that vulnerable / underserved communities are not disproportionately aff
traffic_impact.py	Estimates traffic rerouting costs, congestion buildup, and vehicle-hours of delay caused by disruptions.
temporal_degradation.py	Models how road quality degrades over time without maintenance.
timeline.py	Tracks the event timeline of a simulated disaster for step-by-step replay.
recommendations.py	Generates automated recommendations: priority repairs, new road construction, traffic rerouting suggestion

6.3 External Integrations

Module	Description
bhuvan.py	Proxies satellite imagery tiles from ISRO's Bhuvan geoportal for map overlays.
groq_client.py	Connects to the Groq API for LLM-powered natural-language Q&A (Copilot feature). Supports Llama and Mixtr
nominatim.py	Geocoding: converts place names to coordinates and vice versa via OpenStreetMap Nominatim.
overpass.py	Queries the Overpass API for raw OpenStreetMap data (roads, buildings, POIs) within the area of interest.

6.4 Machine Learning Pipeline

An **optional** ML pipeline (enabled via `ENABLE_ML=true`) provides satellite-image-based road extraction. It is disabled by default because it requires a trained model checkpoint and GPU resources.

Module	Description
model.py	Defines the U-Net architecture (encoder–decoder with skip connections) for semantic segmentation of road pix
inference.py	Loads a trained checkpoint and runs inference on new satellite tiles, producing binary road masks.
train.py	Training loop with configurable epochs, learning rate, augmentations, and validation split.
augmentations.py	Data augmentation pipeline: random flips, rotations, colour jitter, Gaussian blur for training robustness.
explain.py	Grad-CAM heatmaps showing which image regions the model focuses on for road detection — aids interpretab
change_detection.py	Compares road masks from two time periods to detect new construction or road damage.

6.5 Report Generator

The `reports/generator.py` module produces downloadable PDF and HTML reports summarizing the current state of the network, criticality rankings, simulation results, and intervention recommendations. Reports are generated on demand via the `/reports/generate` API endpoint.

7. Frontend — User Interface Pages

The frontend is built with **Next.js 14** using the App Router. It provides 7 main pages, each connected to the backend API. The UI uses **shadcn/ui** components (Radix UI primitives styled with Tailwind CSS) for a consistent, modern look.

Page	Description
/ (Dashboard)	Home page showing a high-level overview: network health score, total nodes and edges, top critical junctions, qui
/map (Heatmap)	Interactive Leaflet map overlaying a criticality heatmap on the road network. Nodes are colour-coded by their com
/simulate	Disaster simulation interface. Users select a scenario (earthquake, flood, targeted attack, custom), configure para
/analytics	Charts and visualisations: centrality distributions (histograms), degree distribution (log-log plot), resilience curves,
/copilot	Chat interface for the AI Copilot. Users ask natural-language questions about the network (e.g., "Which junction, i
/reports	Report generation page. Users can configure report parameters and generate downloadable PDF/HTML resilienc
/explain	ML explanation page (when ML is enabled). Shows segmentation masks overlaid on satellite imagery, Grad-CAM

8. API Reference

The backend exposes **32+ RESTful endpoints** organised under these routers. All endpoints return JSON.

Method	Endpoint	Description
GET	/graph/metrics	Global network metrics (efficiency, connectivity, etc.)
GET	/graph/centrality	Node-level centrality scores
GET	/graph/critical-junctions	Top-N most critical junctions
GET	/graph/geojson	Full graph as GeoJSON (nodes + edges)
GET	/graph/summary	Summary statistics: node/edge counts, bounding box
GET	/graph/mst	Minimum Spanning Tree of the graph
GET	/graph/fingerprint	Graph topology fingerprint
GET	/graph/components	Connected component analysis
GET	/graph/bridges	Bridge edges (whose removal disconnects components)
POST	/simulate/scenario	Run a named disaster scenario
POST	/simulate/cascade	Run cascade-failure simulation
POST	/simulate/ablation	Remove specific nodes/edges and measure impact
POST	/simulate/resilience	Compute resilience score for a disruption
POST	/simulate/evacuation	Plan evacuation routes
POST	/simulate/equity	Equity-weighted disruption analysis
POST	/simulate/traffic-impact	Traffic rerouting cost estimation
POST	/simulate/temporal	Temporal degradation simulation
POST	/simulate/recommendations	Get repair / intervention recommendations
GET	/simulate/scenarios	List available predefined scenarios
GET	/accessibility/isochrone	Travel-time isochrone from a given node
GET	/accessibility/score	Accessibility score for a location
POST	/copilot/ask	Send a natural-language question to the AI copilot
GET	/copilot/context	Retrieve current graph context for the LLM
POST	/reports/generate	Generate a PDF/HTML resilience report
GET	/reports/list	List previously generated reports
GET	/bhuvan/tile/{z}/{x}/{y}	Proxy ISRO Bhuvan satellite tiles
GET	/bhuvan/dem	Fetch DEM elevation data for the AOI

Method	Endpoint	Description
POST	/segment/predict	Run road segmentation on an uploaded image
POST	/segment/explain	Generate Grad-CAM explanation for a segmentation
POST	/segment/change	Detect road changes between two images
GET	/health	Health check — returns {status: 'ok'}
GET	/	Root — returns welcome message

9. Graph Metrics & Key Statistics

When loaded with the default Bengaluru AOI (Area of Interest), the graph has the following characteristics:

Metric	Value
Total Nodes (Intersections)	13,486
Total Edges (Road Segments)	19,117
Graph Type	MultiDiGraph (directed, multi-edge)
Coordinate System	WGS 84 (EPSG:4326)
Average Node Degree	~2.84
Network Diameter	Computed dynamically on largest connected component

Resilience Metrics Explained

Metric	Meaning
Global Efficiency	Average inverse shortest-path length between all node pairs. Higher = better connected. Ranges 0–1.
Algebraic Connectivity	Second-smallest eigenvalue of the Laplacian. Measures how hard it is to partition the graph. Higher = r
MST Fraction	Ratio of MST weight to total edge weight. Lower = more redundancy (good for resilience).
Average Clustering	Mean local clustering coefficient. Higher = more triangles / alternate routes at the local level.
Connected Components	Number of disconnected subgraphs. Ideally 1 (fully connected). Increases after disruptions.
Betweenness Centrality	Fraction of shortest paths passing through a node. High betweenness = critical bottleneck.
Closeness Centrality	Inverse of the sum of shortest distances from a node to all others. High closeness = well-positioned.

10. Disaster Simulation Engine

The simulation engine is the analytical core of the system. It models various disaster scenarios and quantifies their impact through multiple lenses.

10.1 Scenario Types

Scenario	Mechanism
Earthquake	Removes nodes/edges within a radius of the epicentre based on seismic intensity and fragility curves.
Flood	Disables low-elevation road segments (below a configurable water level) using DEM data.
Landslide	Targets steep-gradient road segments identified from elevation profiles.
Targeted Attack	Removes the top-N most critical nodes (by betweenness centrality) to simulate worst-case deliberate disruption.
Random Failure	Randomly removes a percentage of nodes/edges to model stochastic infrastructure decay.
Custom	User-defined: select specific nodes/edges or draw a polygon on the map.

10.2 Simulation Outputs

- **Before / After Metrics:** Global efficiency, algebraic connectivity, connected components — compared side-by-side.
- **Fragmentation Map:** GeoJSON of disconnected components, colour-coded by size.
- **Detour Analysis:** For critical OD pairs, the increase in travel time/distance.
- **Population Impact:** Estimated number of people in isolated network fragments.
- **Cascade Propagation:** Step-by-step timeline of which nodes fail in sequence.
- **Recommendations:** Prioritised list of edges to add/repair for maximum resilience recovery.

10.3 Counterfactual Interventions

After a simulation, the system can propose **healing interventions**:

- MST-based reconnection of fragmented components.
- Bridge / bypass road insertion at optimal locations.
- Each proposed intervention includes a cost estimate (road length) and resilience gain (Δ efficiency, Δ connectivity).

11. ML / AI Components

11.1 Road Segmentation (U-Net)

The optional ML pipeline uses a **U-Net** encoder-decoder architecture for semantic segmentation of road pixels in satellite imagery. The encoder uses a pretrained ResNet backbone, while the decoder upsamples feature maps with skip connections to produce a pixel-wise binary mask (road vs. non-road).

- Input: 256×256 satellite image tiles
- Output: Binary segmentation mask
- Loss: Binary cross-entropy + Dice loss
- Augmentations: Random flips, rotations, colour jitter, Gaussian blur

11.2 Grad-CAM Explanations

To improve interpretability, Grad-CAM (Gradient-weighted Class Activation Mapping) generates attention heatmaps showing which regions of the input image most influence the model's road predictions. Available via the `/explain` page.

11.3 Change Detection

By comparing segmentation masks from two time periods (e.g., before and after a disaster), the system detects **new road construction** or **road damage/destruction**. Output is a change map with three classes: unchanged, added, removed.

11.4 AI Copilot (Groq LLM)

The Copilot feature connects to the **Groq API** (running Llama or Mixtral models) to provide natural-language interaction. The system injects the current graph context (metrics, top critical nodes, recent simulation results) into the LLM prompt, enabling context-aware Q&A; such as:

- "Which junction, if removed, causes the biggest efficiency drop?"
- "How many connected components exist after removing the top 5 critical nodes?"
- "Suggest the best location for a new bypass road."

12. External Integrations

Service	Purpose
OpenStreetMap / Overpass	Primary source of road network data. OSMnx queries the Overpass API to download road graphs for any b
ISRO Bhuvan	Satellite imagery tile server. The backend proxies tiles from Bhuvan's WMS for map overlays in the fronten
Nominatim	Geocoding service for converting place names ↔ coordinates.
Groq API	Cloud LLM inference for the AI Copilot. Requires a GROQ_API_KEY.
WorldPop	Population density raster data for estimating population impact of disruptions.

13. Feature Coverage Matrix

Status of all major features in the current build:

Feature	Status	Notes
OSM Graph Ingestion	■ Working	Loads from Overpass cache or live API
Centrality Computation	■ Working	Betweenness, closeness, degree, eigenvector
Global Metrics	■ Working	Efficiency, algebraic connectivity, clustering
Critical Junction Detection	■ Working	Composite ranking from multiple centrality measures
Interactive Map (Leaflet)	■ Working	Heatmap overlay, layer toggles
Disaster Simulation	■ Working	Multiple scenario types supported
Cascade Failure Model	■ Working	Chain-reaction node failure simulation
Counterfactual Healing (MST)	■ Working	Propose reconnection edges
AI Copilot (Groq LLM)	■■ Needs Key	Requires GROQ_API_KEY in .env
Report Generation	■ Working	PDF/HTML reports via /reports/generate
U-Net Road Segmentation	■■ Optional	Requires ENABLE_ML=true + trained model
Grad-CAM Explanations	■■ Optional	Requires ML pipeline enabled
Change Detection	■■ Optional	Requires two temporal image sets
Population Impact Analysis	■■ Needs Data	Requires WorldPop raster download
DEM / Topography Analysis	■■ Needs Data	Requires DEM tiles download
Bhuvan Tile Proxy	■ Working	Proxies ISRO satellite imagery tiles
Evacuation Planning	■ Working	Route planning to safe zones
Equity Analysis	■ Working	Vulnerability-weighted impact assessment

14. Setup & Quick Start Guide

Prerequisites

- Python 3.10+ installed and on PATH
- Node.js 18+ and npm installed
- Git installed

Step 1: Clone the Repository

```
git clone https://github.com/<your-username>/Route-Resilience-System.git
cd Route-Resilience-System
```

Step 2: Backend Setup

```
cd backend
python -m venv .venv
.venv\Scripts\activate # Windows
# source .venv/bin/activate # Linux/macOS
pip install -r requirements-core.txt
```

Step 3: Configure Environment

```
copy .env.example .env
```

Edit `.env` and set at minimum:

```
AOI_BBOX=12.9,77.5,13.1,77.7 # Bengaluru bounding box
GROQ_API_KEY=gsk_... # Optional: for AI Copilot
ENABLE_ML=false # Set true for ML pipeline
```

Step 4: Start Backend

```
cd backend
uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Backend starts at `http://127.0.0.1:8000`. On first run, it downloads and caches the road graph (~30 seconds).

Step 5: Frontend Setup

```
cd frontend
npm install
npm run dev
```

Frontend starts at `http://localhost:3000`.

Step 6: Open the Application

Navigate to `http://localhost:3000` in your browser. The dashboard loads automatically and displays network health metrics.

15. Environment Variables

All configuration is via a `.env` file in the project root:

Variable	Description	Default
AOI_BBOX	Bounding box (south, west, north, east) for the area of interest	12.9,77.5,13.1,77.7
GROQ_API_KEY	API key for Groq LLM (enables AI Copilot)	(none)
ENABLE_ML	Enable the ML segmentation pipeline	false
BACKEND_URL	Backend URL for the frontend to connect to	http://127.0.0.1:8000
MODEL_PATH	Path to the trained U-Net model checkpoint	models/best.pt
DEM_PATH	Path to DEM (elevation) raster file	data/dem/
WORLDPOP_PATH	Path to WorldPop population raster	data/worldpop/

16. Future Scope

- Real-time traffic data integration for dynamic edge weighting.
- Multi-city support with a city-selector UI.
- Live satellite imagery processing for real-time damage assessment.
- Integration with ISRO's Bhuvan APIs for higher-resolution imagery.
- Mobile-responsive frontend for field teams.
- Historical disaster replay: load past events and validate model predictions.
- Collaborative annotation: allow multiple users to mark road damage on the map.
- Containerisation with Docker / Docker Compose for one-command deployment.
- CI/CD pipeline with automated testing and deployment.

— End of Document —